

BIT 4107 – Mobile Application Development

ENHANCED PROFESSIONAL STUDENT LOGBOOK

Student Name: Pheneas maina

Admission Number: BIT/2024/59093

School/Faculty: Computing And Informatics

Programme:

Academic Year: 2026

Semester: sem 7

Lecturer: Mr Nyoro

Phone Number: 0700232122

Instructions

This logbook is used to document weekly learning activities, practical exercises, skills acquired, challenges encountered, solutions applied, and reflections. Students should complete all sections every week.

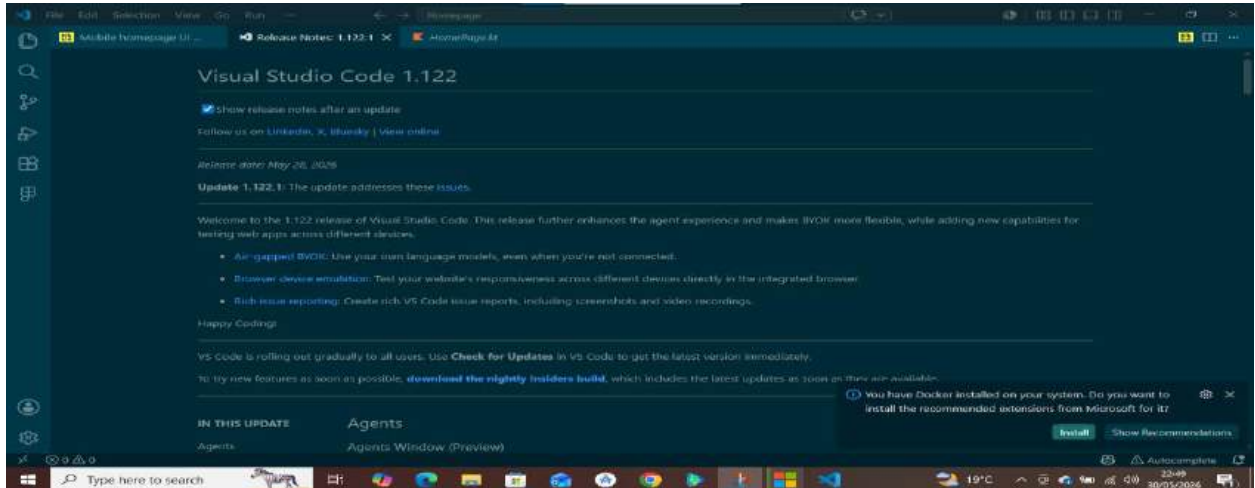
Each week to follow remarks in week 1 section.

Autogenerated Table of content

Week 1: Introduction to Mobile Application Development

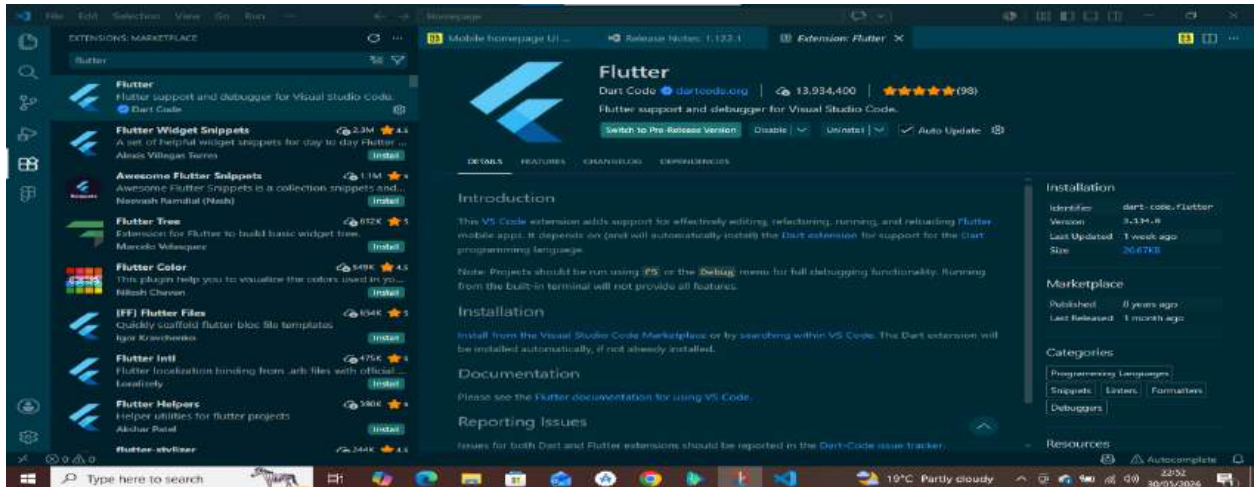
Development of an Interactive Flutter "Hello World" App

1. Installation of Visual Studio Code (VS Code)



- Downloaded and installed Visual Studio Code version 1.122 as shown in the update screenshot.
- During setup, the system prompted to install recommended Docker extensions, but this was optional for Flutter development.

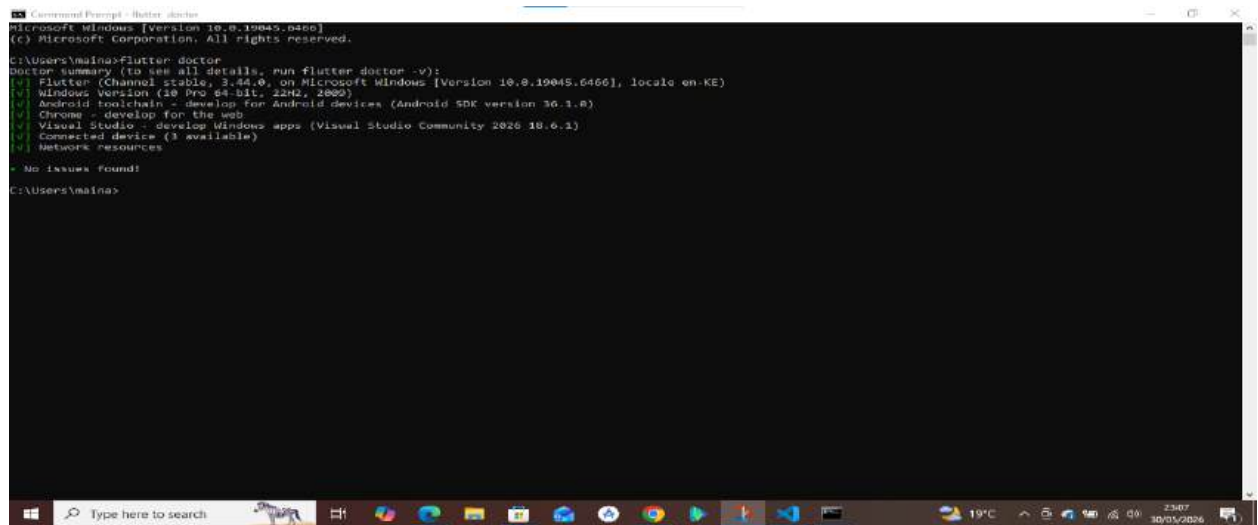
2. Installing Flutter and Dart Extensions in VS Code



- Opened VS Code and navigated to the Extensions Marketplace.
- Searched for and installed the official Flutter extension (by Dart Code, version 3.134.0). The Dart extension was installed automatically as a dependency.

- Additional helpful extensions were available (e.g., Flutter Widget Snippets, Flutter Tree), but the core requirement was the official Flutter/Dart support.

3. Verifying the Development Environment using Flutter Doctor



```
Microsoft Windows [version 10.0.19045.0466]
(c) Microsoft Corporation. All rights reserved.

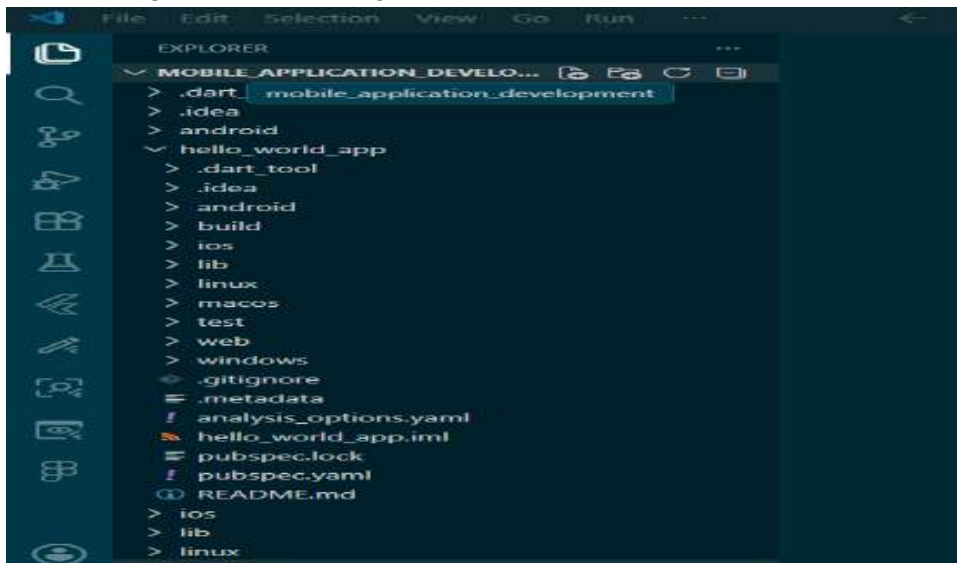
C:\Users\maina>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.44.0, on Microsoft Windows [Version 10.0.19045.0466], locale en-KE)
[✓] Windows Version (10 Pro 64-bit, 22H2, 2000)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2026 18.0.1)
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!

C:\Users\maina>
```

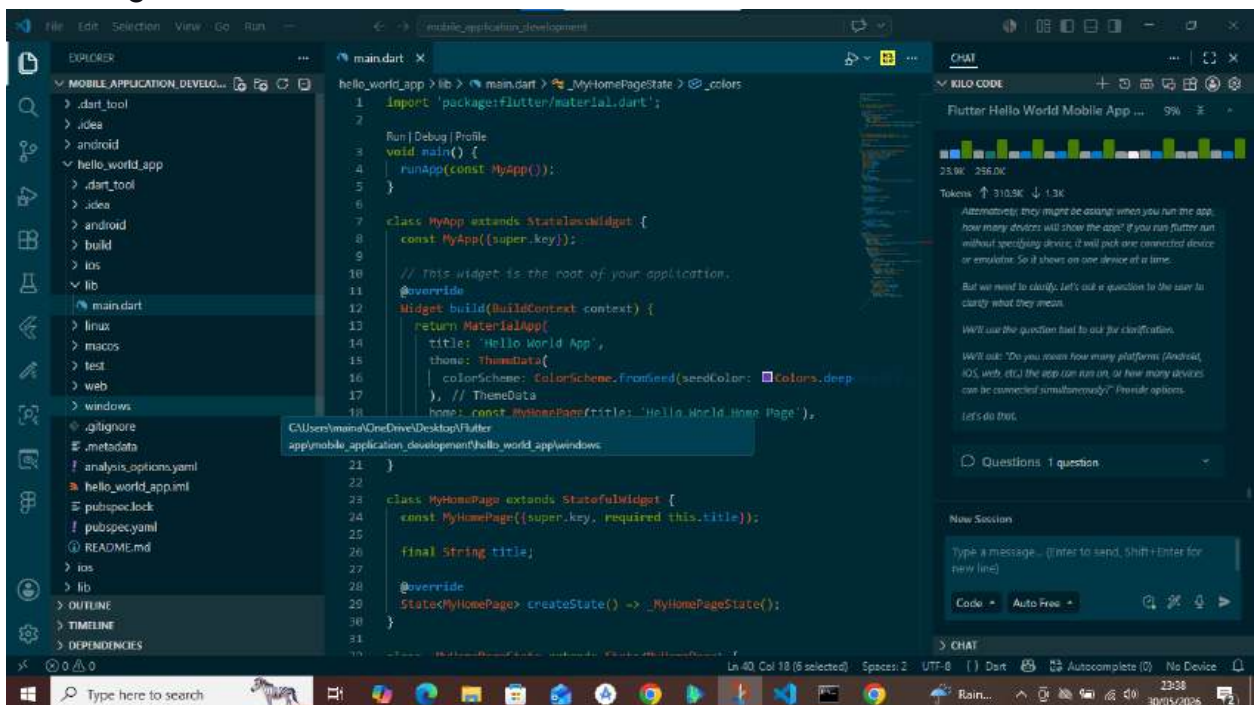
- Opened Command Prompt (cmd) as shown
- Ran the command: `flutter doctor`
- The output confirmed no issues:
 - Flutter SDK (stable channel, version 3.44.0) was installed.
 - Chrome and Edge were available for web testing.
 - Android toolchain and Visual Studio for Windows development were correctly configured.
 - Connected devices: Windows, Chrome, and Edge were detected.

4. Creating the Flutter Project



- In VS Code, created a new Flutter project named hello_world_app.
- The project location was: C:\Users\mamina\OneDrive\Desktop\Flutter app\mobile_application_development\hello_world_app

5. Writing the Source Code



- Developed a StatefulWidget to manage interactive elements.

- Key features implemented:
 - A TextField for user input.
 - A Text widget to display output.
 - A button that, when clicked, shows the typed text.
 - A second button that randomly changes the background color of the app bar or container.

6. Running the Application

```

C:\Windows\System32\cmd.exe - flutter run - flutter run
Waiting for connection from debug service on Chrome... 52.0s
Failed to compile application.

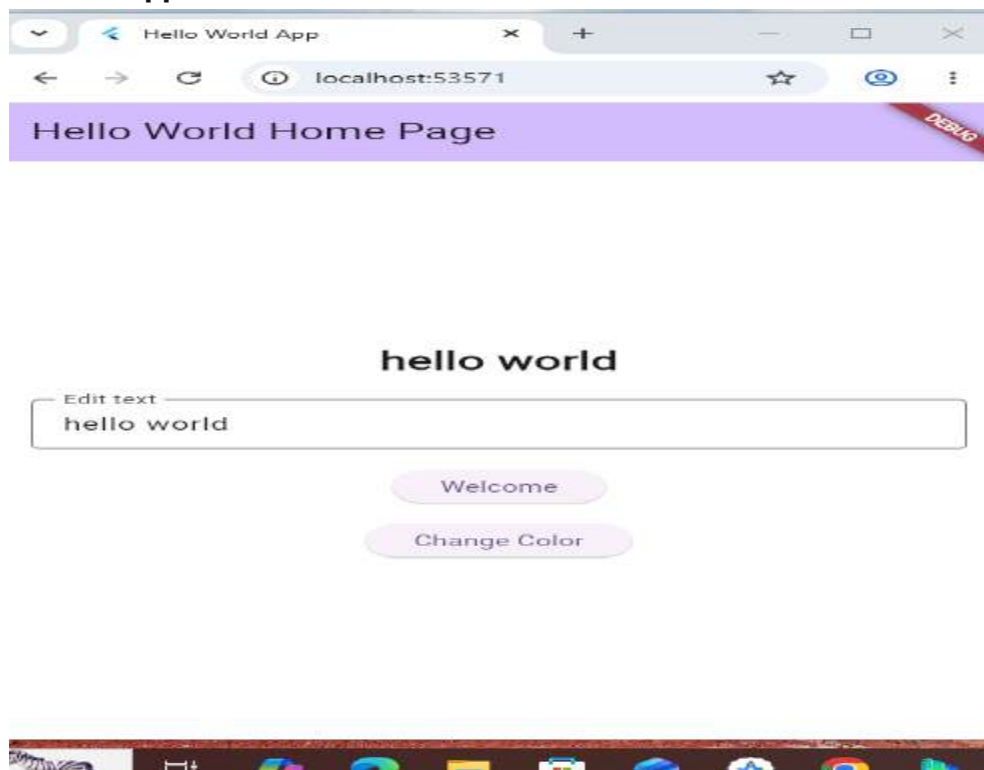
C:\Users\maina\OneDrive\Desktop\Flutter app\mobile_application_development\hello_world_app>flutter run
Connected devices:
Windows (desktop) • windows • windows.x64 • Microsoft Windows [version 10.0.19045.6466]
Chrome (web) • chrome • web-javascript • Google Chrome 148.0.7778.181
Edge (web) • edge • web-javascript • Microsoft Edge 148.0.3967.96
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (or "q" to quit): 2
Launching lib/main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome... 43.0s

Flutter run key commands:
h Hot reload.
R Hot restart.
d List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

This app is linked to the debug service: ws://127.0.0.1:53720/PVzerv-RXOs-/ws
Debug service listening on ws://127.0.0.1:53720/PVzerv-RXOs-/ws
A Dart VM Service on Chrome is available at: http://127.0.0.1:53720/PVzerv-RXOs-
The Flutter DevTools debugger and profiler on Chrome is available at:
http://127.0.0.1:53720/PVzerv-RXOs-/devtools/?uri=ws://127.0.0.1:53720/PVzerv-RXOs-/ws
Starting application from main method in: org-dartlang-appi/web_entrypoint.dart.
  
```

- In the terminal (cmd or VS Code terminal), ran the command: `flutter run`
- The system detected multiple devices: Windows, Chrome, and Edge.
- Selected Chrome (Option 2) for web testing.
- The app compiled and launched successfully, connecting to a debug service on `ws://127.0.0.1:53720`.

7. Final App Behavior



- The app displayed a "Hello World Home Page".
- A text box allowed typing (e.g., "hello world").
- Button 1 (Change Color): Clicking this button changed the color theme of the app (e.g., AppBar or background color).
- Button 2 (Edit text / Welcome): Clicking this button displayed the user's typed text on the screen

I Successfully set up a Flutter development environment in VS Code, created a project, wrote interactive code with input/output and color-changing functionality, and ran it on a Chrome web browser as my emulator with full debug capabilities.

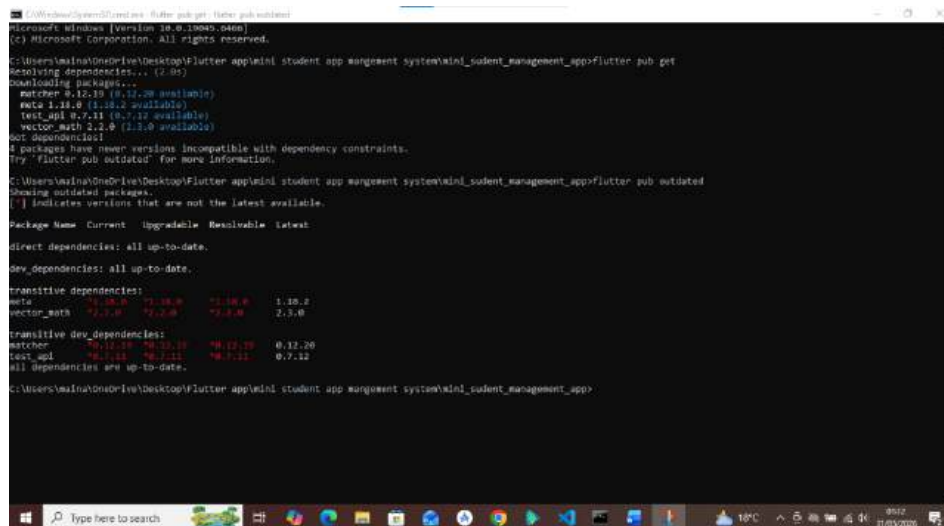
Week 2: Mobile Development Environment

1. PROJECT SETUP

The project was initiated from a default Flutter counter app generated by the `flutter create` command.

- Existing file: `lib/main.dart`
- Existing file: `pubspec.yaml`

2. DEPENDENCIES ADDED



```
C:\Users\malina\OneDrive\Desktop\Flutter app\mini student app management system\mini_student_management_app>flutter pub get
Microsoft Windows [version 10.0.19040.0]
(c) Microsoft Corporation. All rights reserved.

C:\Users\malina\OneDrive\Desktop\Flutter app\mini student app management system\mini_student_management_app>flutter pub get
Resolving dependencies... (2.0s)
Downloading packages...
  matcher 0.12.13 (0.12.20 available)
  meta 1.10.0 (1.10.2 available)
  test_api 0.7.11 (0.7.12 available)
  vector_math 2.1.0 (2.1.0 available)
Got dependencies!
4 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.

C:\Users\malina\OneDrive\Desktop\Flutter app\mini student app management system\mini_student_management_app>flutter pub outdated
Showing outdated packages.
[!] indicates versions that are not the latest available.

Package Name  Current  Upgradable  Resolvable  Latest

direct dependencies: all up-to-date.
dev_dependencies: all up-to-date.

transitive dependencies:
meta          ^1.10.0  ^1.10.0     ^1.10.0     1.10.2
vector_math   ^2.1.0   ^2.1.0     ^2.1.0     2.1.0

transitive dev_dependencies:
matcher        ^0.12.10 ^0.12.10   ^0.12.10   0.12.20
test_api       ^0.7.10  ^0.7.11    ^0.7.11    0.7.12
all dependencies are up-to-date.

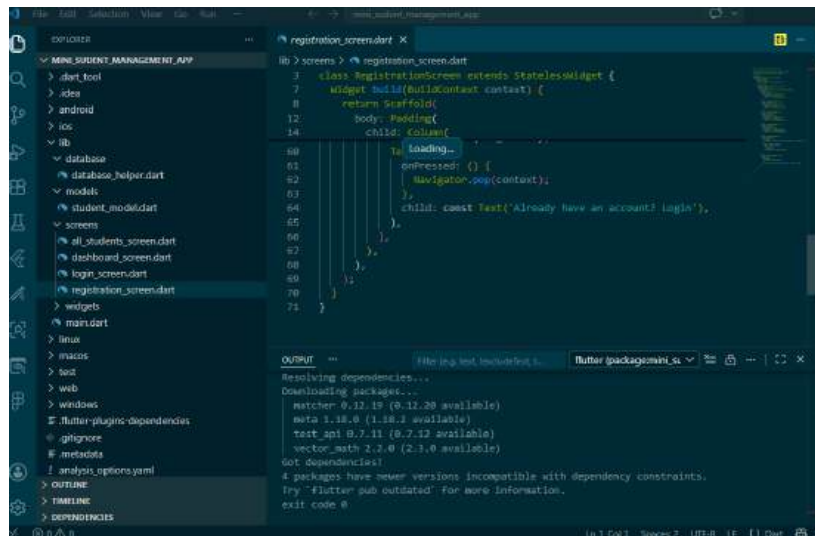
C:\Users\malina\OneDrive\Desktop\Flutter app\mini student app management system\mini_student_management_app>
```

The following packages were added to `pubspec.yaml`:

- `sqlite3: ^2.3.0`
- `path_provider: ^2.1.0`
- `path: ^1.8.0`

Verification: The file `.dart_tool/package_config.json` confirmed that all packages were downloaded successfully.

3. PROJECT STRUCTURE CREATED



The following directory and file structure was established within the lib/ folder:

text

lib/

main.dart

screens/

login_screen.dart

registration_screen.dart

dashboard_screen.dart

all_students_screen.dart

models/

student_model.dart

database/

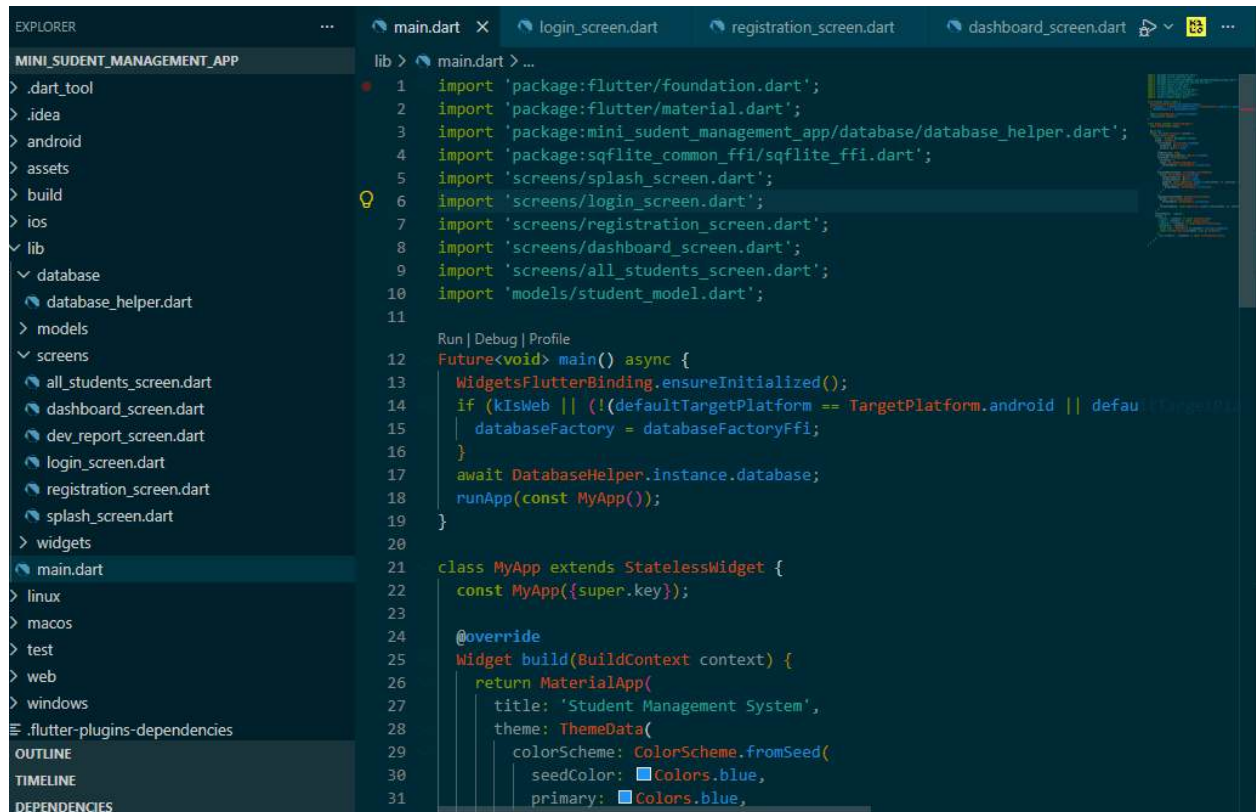
database_helper.dart

widgets/

custom_button.dart

custom_textfield.dart

4. MAIN.DART CONFIGURATION

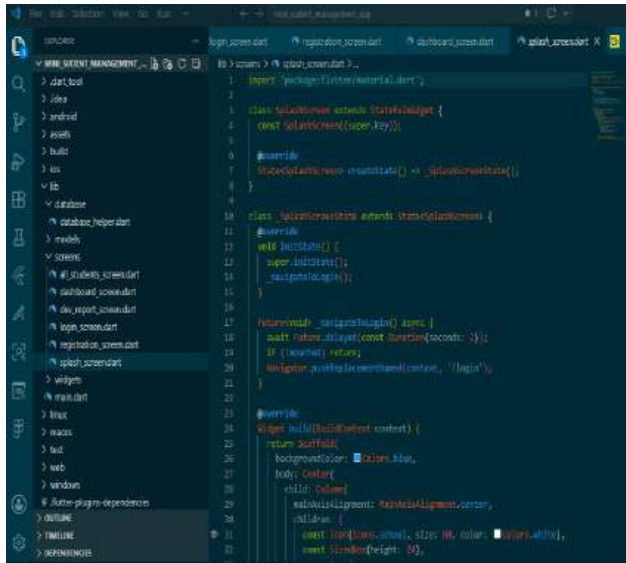


The default counter app in `main.dart` was completely replaced with a new configuration.

- **SQLite Initialization:** Added `WidgetsFlutterBinding.ensureInitialized()` and `await DatabaseHelper.instance.database` before `runApp()` to ensure the database is ready on startup.
- **Theme:** Configured `MaterialApp` with Material 3, setting the primary color to `Colors.blue` and the scaffold background to `Colors.grey[50]`. Global styles were applied for rounded cards (radius 12) and rounded buttons (radius 8).
- **Routes:** Defined named routes for navigation:
 - `/splash` (initial route)
 - `/login`
 - `/register`
 - `/dashboard`
 - `/all_students`
- **Data Passing:** The dashboard route is configured to accept a `Student` object via `ModalRoute.settings.arguments`.

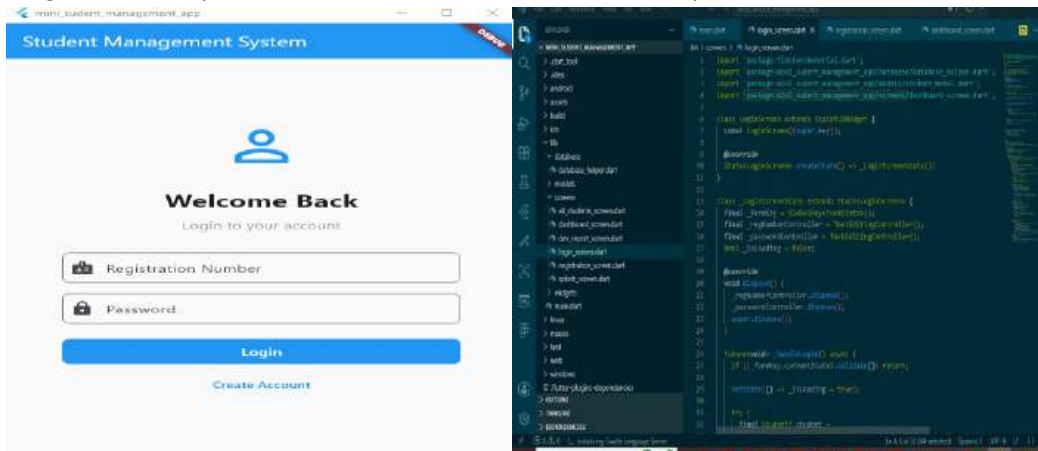
5. SCREEN DETAILS

SplashScreen (lib/screens/splash_screen.dart)



A new file created for the app launch experience. It displays a blue background with a school icon and app title for 2 seconds, along with a CircularProgressIndicator, before navigating to the /login screen.

LoginScreen (lib/screens/login_screen.dart)

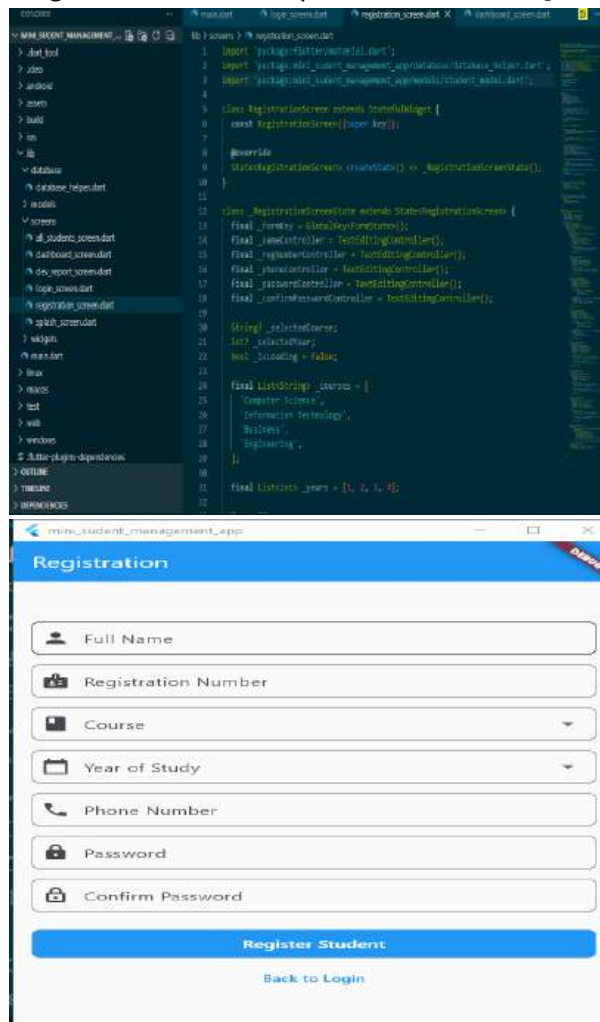


A stateful widget for user authentication.

- UI: Includes a blue AppBar, a responsive layout capped at 400px, and text fields for Registration Number and Password with form validation.
- Logic: Uses DatabaseHelper.instance.getStudentByRegNumber() to verify credentials. Displays specific error messages for "Registration number not found" and "Incorrect password."

- Navigation: Passes the authenticated Student object to the DashboardScreen on success. A "Create Account" button navigates to the registration screen.

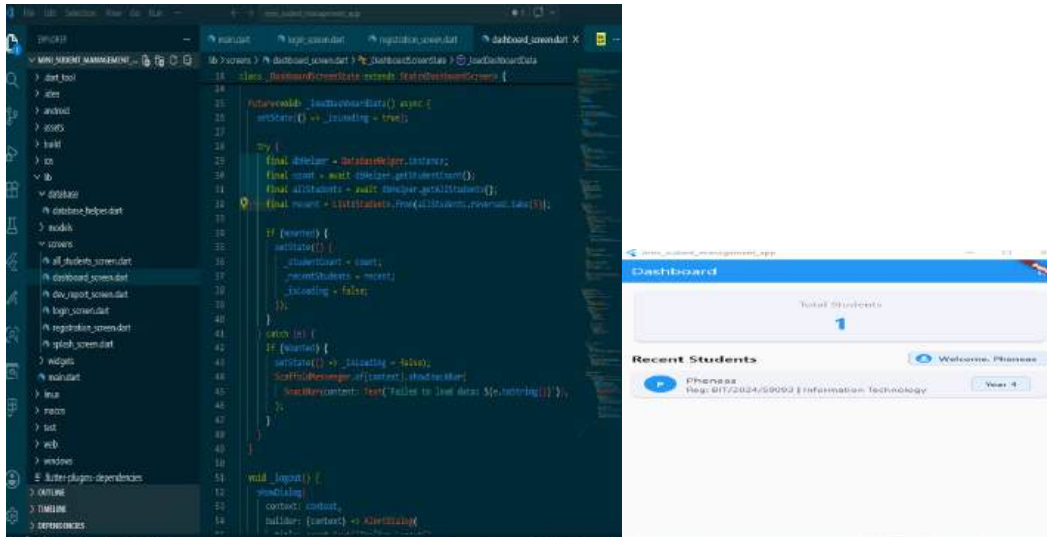
RegistrationScreen (lib/screens/registration_screen.dart)



A stateful widget for creating new student accounts.

- UI: Features a blue AppBar and a responsive layout capped at 500px. Contains fields for Full Name, Registration Number, Phone (with 10-digit validation), Password, and Confirm Password, along with dropdowns for Course and Year of Study.
- Logic: On submission, it checks for duplicate registration numbers, creates a new Student object, inserts it into the database, and navigates to the login screen. A "Back to Login" button is also provided.

DashboardScreen (lib/screens/dashboard_screen.dart)



The main hub for a logged-in user.

- **UI:** Displays a personalized welcome message. The AppBar has no back button but includes a logout icon with a confirmation dialog. A navigation drawer provides links to "All Students", "Add New Student", and "Logout".
- **Data:** Shows a card with the total student count and a list of the last 5 registered students, each displaying a name initial avatar, registration number, course, and year.
- **Interaction:** Supports pull-to-refresh to reload data.

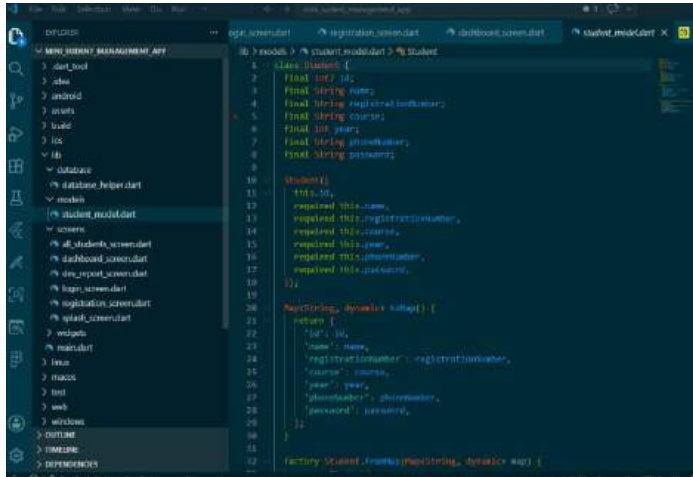
AllStudentsScreen (lib/screens/all_students_screen.dart)

A stateful widget displaying the complete list of students.

- **UI:** Contains a blue AppBar and a FloatingActionButton to add new students.
- **List View:** Shows each student with an initial avatar, name, and a subtitle with their registration number, course, and year.
- **Edit:** Tapping a student tile opens an alert dialog pre-filled with their data, allowing updates to the name, course, year, and phone number.
- **Delete:** A red delete icon on each tile triggers a confirmation dialog. Upon confirmation, the student record is deleted.

- Interaction: Supports pull-to-refresh.

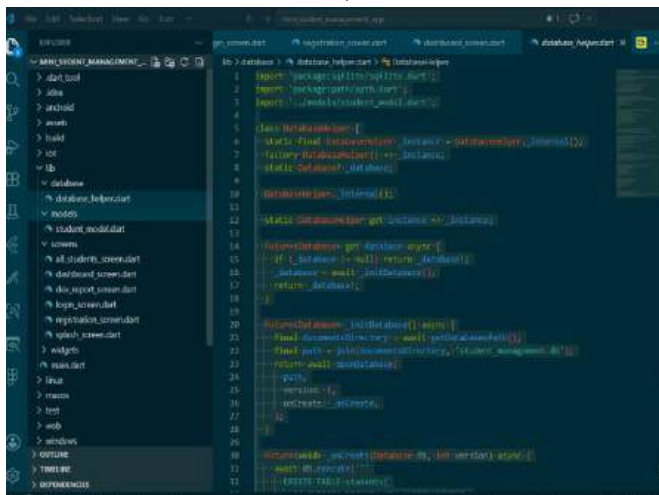
6. MODEL: STUDENT (lib/models/student_model.dart)



Defines the data structure for a student with the following fields: id, name, registrationNumber, course, year, phoneNumber, and password. It includes:

- A constructor with named parameters.
- A `toMap()` method to convert a `Student` object to a map for the database.
- A `fromMap()` factory method to create a `Student` object from a database map.

7. DATABASE HELPER (lib/database/database_helper.dart)



Manages the SQLite database using the singleton pattern.

- Database: Initializes a database named `student_management.db` and creates a `students` table with columns for ID, name, registration number, course, year, phone number, and password.
- Methods: Provides the following functions for CRUD operations:
 - `insertStudent`
 - `getStudentByRegNumber`
 - `getAllStudents`
 - `updateStudent`
 - `deleteStudent`
 - `getStudentCount`

Week 3: User Interface Design

COMPLETE UI PROTOTYPE

APP OVERVIEW

App Name: Daylight Restaurant

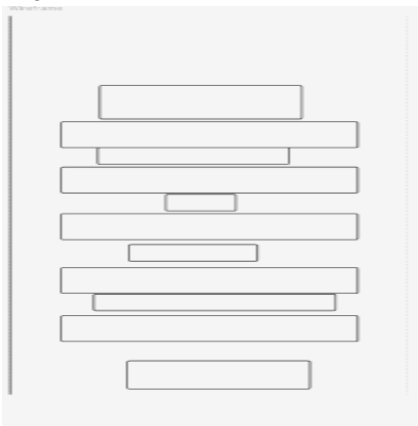
Target Users: Customers

WIREFRAMES (FIGMA)

I created low-fidelity wireframes for all screens in Figma focusing on layout and navigation flow.

Wireframe Screenshots:

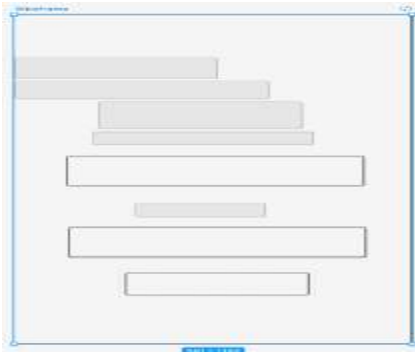
Registration Screen



Home Screen



Log in page



menu page wireframe

Reservation page wireframe



Figma File Link:

<https://www.figma.com/design/G0PbVDNwQ9Mj6PU4HQjtJZ/Untitled?node-id=0-1&t=goWjyztAzPmzw6SC-1>

HIGH-FIDELITY GUI DESIGN

After wireframes , I applied colors, typography, and spacing to create the final designs.

Final GUI Screenshots:

Login Screen



Registration Screen

Home page

Sign up page

Premium Steakhouse & grill

Sign up

username /email

Phone number

email

Password

Confirm Password

Sign up

Homepage

Daylight

Premium Steakhouse & grill

Our speciality

Reserve

Exclusively serving the finest cuts of beef, lamb, and premium meats. All our steaks are dry-aged in house for exceptional flavor

Michelin rated Master chef Top rated

Featured Dishes

View all

	Prime ribeye	Ksh 1500
Aged 28 days, served with seasonal vegetables		
	Prime ribeye	Ksh 1500
Aged 28 days, served with seasonal vegetables		
	Wagyu Tenderloin	Ksh 1500
Aged 28 days, served with seasonal vegetables		
	Tomahawk steak	Ksh 5000
Aged 28 days, served with seasonal vegetables		

Menu page

Menu

Our menu

Search dishes...

All beef lamb special

	Prime ribeye	Ksh 1500
Aged 28 days, served with seasonal vegetables		
	Prime ribeye	Ksh 1500
Aged 28 days, served with seasonal vegetables		
	Wagyu Tenderloin	Ksh 1500
Aged 28 days, served with seasonal vegetables		
	Tomahawk steak	Ksh 5000
Aged 28 days, served with seasonal vegetables		
	Beef steak	Ksh 5000
Aged 28 days, served with seasonal vegetables		
	Goat meat	Ksh 5000
Aged 28 days, served with seasonal vegetables		

Booking page

Reserve your table

Book a table

Reserve your dining experience

Number of guests

Date

Select Time

Confirm reservation

COLOR SCHEME & TYPOGRAPHY

Colors:

- Primary: Orange and black – Buttons and headers
- Background: White – Main screen

Typography:

- Headings 24px, Bold
- Body: 14px, raven
- Buttons: 14px, bold

CODE IMPLEMENTATION

Project Setup Command:

```
Kotlin
```

Key Code – Main App:

Code is connected to my github account Phin489

SCREENS & NAVIGATION

Five Screens Implemented:

1. Login Screen – Email and password entry
2. Registration Screen – New account creation
3. Home/Dashboard – Main content display
4. Menu screen–contains food in the restaurant
5. Booking page – People can book table or services

FORM VALIDATION

Validation Rules:

- Email must contain @ symbol
- Password minimum 6 characters
- Confirm password must match password
- Required fields cannot be empty



RESPONSIVE TESTING

Tested on:

- Phone emulator (Pixel 5) – Working



CONCLUSION

I completed all requirements: 5 screens, working navigation, responsive design, functional forms with validation, and modern UI implemented in both Figma and code.

GitHub Repository: <https://github.com/Phin489/week-three-Daylight-restaurant->

Week 4: Event Handling and User Interaction

1. Text Input Handling (Forms & Authentication)

When a user types into a text field (like the login or sign-up screens), it uses an `OutlinedTextField`



2. Action Clicks (Buttons & Submissions)

When a user submits a form or confirms an action, they interact with standard `Button` or `TextButton` components

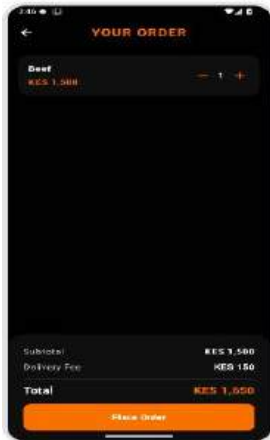
3. Navigation Interactions

Navigating between screens (like moving from the Home screen to the Menu or the Admin Dashboard) is also handled via click events.



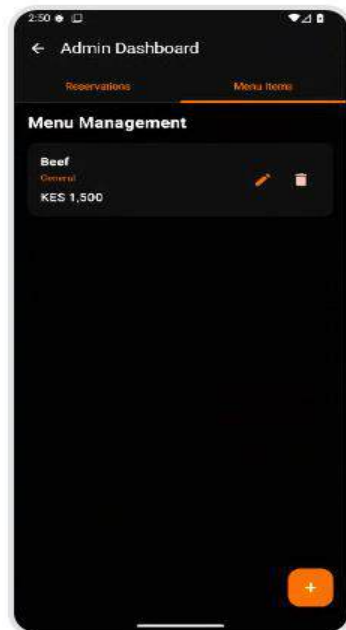
4. Interactive Lists (Cart & Admin Adjustments)

For lists containing multiple items—such as adding/subtracting items in the Cart or editing menu items in the Admin Dashboard—events are lifted up from the item level.



5. Dialogs and Modals

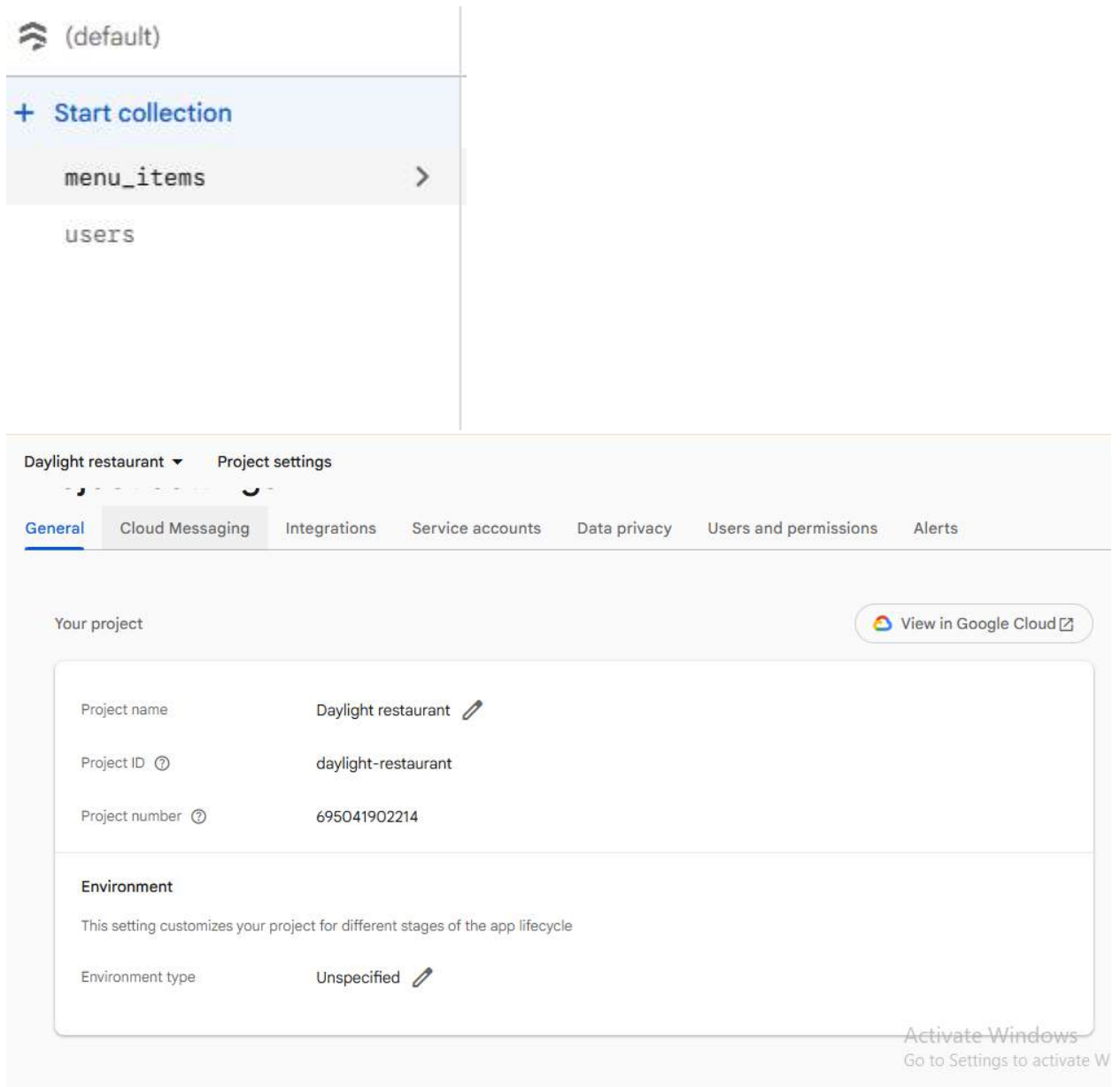
When an Admin wants to add a new item or edit a reservation, they are presented with an AlertDialog overlay.



Week 5: Data Management

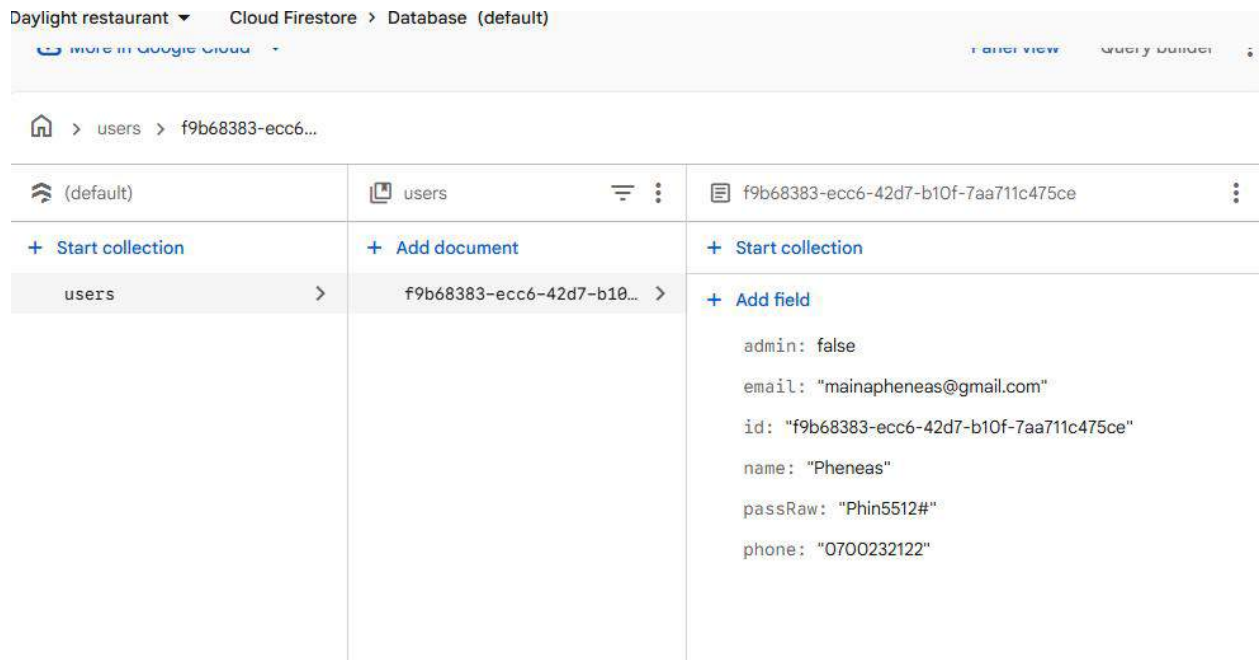
Data management in the app is handled through a modern, real-time cloud architecture using **Firebase Cloud Firestore** and **Kotlin Flows**.

1. **Cloud Database (Firestore):** Instead of a local offline database, the app stores all data in Firebase Firestore collections (users, menu_items, and reservations). This acts as the single source of truth.



2. **Data Models:** Data is structured into clean data classes that easily map to Firestore documents.
3. **Repository Layer (AppRepository):** This is the central hub for data operations. It connects to Firestore and sets up real-time snapshot listeners. Whenever data

changes in the cloud (e.g., an admin updates a menu item, or a user makes a reservation), Firestore pushes the update to the repository instantly.



4. **State Management (MainViewModel):** The repository exposes the data streams using Kotlin StateFlow. The MainViewModel subscribes to these streams and holds the current UI state.
5. **UI Layer:** The Jetpack Compose screens observe the MainViewModel. Because the entire chain is reactive, any change in the cloud database automatically triggers a UI update for the user seamlessly.

Week 6: Networking and APIs

The application utilizes **Firebase Firestore** as its primary cloud database. The network architecture relies on persistent, real-time snapshot listeners that stream data changes via Kotlin Coroutines and StateFlows, rather than relying on traditional one-off HTTP REST calls

1. The Core Network Engine

File: AppRepository.kt

This is the most critical file for network communication. It abstracts the cloud backend away from the rest of the application.

- **Initialization:** It initializes the FirebaseFirestore network client.

```
class AppRepository {  
  
    private val firestore by lazy { FirebaseFirestore.getInstance() }  
  
    private val _menuItems = MutableStateFlow<List<MenuItem>>(emptyList())  
    val menuItems = _menuItems.asStateFlow()  
  
    init {
```

- **Real-time Streaming:** It opens persistent network streams using `.addSnapshotListener()`. Whenever a document is added, modified, or deleted in the cloud the network stream automatically pushes the delta to the app.

```
fun getUserReservations(userId: String): Flow<List<Reservation>> {  
    val reservations = MutableStateFlow<List<Reservation>>(emptyList())  
    firestore.collection("reservations")  
        .whereEqualTo("userId", userId)  
        .addSnapshotListener { snapshot, _ ->  
            if (snapshot != null) {  
                reservations.value = snapshot.documents.mapNotNull { it.toObject(Reservation::class) }  
            }  
        }  
    return reservations.asStateFlow()  
}  
  
fun getAllReservations(): Flow<List<Reservation>> {  
    val reservations = MutableStateFlow<List<Reservation>>(emptyList())  
    firestore.collection("reservations")  
        .addSnapshotListener { snapshot, _ ->  
            if (snapshot != null) {  
                reservations.value = snapshot.documents.mapNotNull { it.toObject(Reservation::class) }  
            }  
        }  
}
```

- **Downstream Bridging:** It converts these proprietary Firebase streams into Kotlin MutableStateFlow, providing a clean, reactive data pipeline to the ViewModel.

- **CRUD Operations:** It handles uploading data suspending the coroutine until the network request successfully completes.

```

suspend fun addMenu(item: MenuItem) {
    val newId = UUID.randomUUID().toString()
    val itemToSave = item.copy(id = newId)
    firestore.collection("menu_items").document(newId).set(itemToSave).await()
}

suspend fun updateMenu(item: MenuItem) {
    if (item.id.isNotEmpty()) {
        firestore.collection("menu_items").document(item.id).set(item).await()
    }
}

suspend fun deleteMenu(item: MenuItem) {
    if (item.id.isNotEmpty()) {
        firestore.collection("menu_items").document(item.id).delete().await()
    }
}

```

Activate Windows
Go to Settings to activate Windows.

2. The State Manager

File: **MainViewModel.kt**

```

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.data.AppRepository
import com.example.model.MenuItem
import com.example.model.Reservation
import com.example.model.User
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

class MainViewModel(private val repo: AppRepository) : ViewModel() {
    private val _uiState = MutableStateFlow(AppState())
    val uiState: StateFlow<AppState> = _uiState
}

```

This file acts as the bridge between the network streams and the user interface.

- **Consumption:** It listens to the StateFlow streams exposed by the AppRepository.
- **Authentication & Lifecycle Binding:** It triggers network actions (like login and register) and observes user-specific reservation streams based on the authenticated

user ID.

```
fun login(email: String, pass: String, onResult: (Boolean) -> Unit) {
    viewModelScope.launch {
        val user = repo.login(email.trim(), pass)
        if (user != null) {
            _uiState.value = _uiState.value.copy(currentUser = user)
            observeUserReservations(user.id)
            if (user.isAdmin) observeAllReservations()
            onResult(true)
        } else {
            onResult(false)
        }
    }
}

fun logout() {
    _uiState.value = _uiState.value.copy(currentUser = null, reservations = emptyList())
}

fun register(name: String, email: String, phone: String, pass: String, onResult: (Boolean) -> Unit) {
    viewModelScope.launch {
        val success = repo.register(User(name = name, email = email, phone = phone, passRaw = pass))
        onResult(success)
    }
}
```

- **UI State Sync:** When new data arrives through the network socket, this file updates the `_uiState`, immediately forcing the Jetpack Compose UI to recompose and display the latest cloud data.

3. Network Configuration & Dependencies

- `app/google-services.json`

```
{
  "project_info": {
    "project_number": "695041902214",
    "project_id": "daylight-restaurant",
    "storage_bucket": "daylight-restaurant.firebaseio.com"
  },
  "client": [
    {
      "client_info": {
        "mobilesdk_app_id": "1:695041902214:android:23a4682a7f602c7364b307",
        "android_client_info": {
          "package_name": "com.aistudio.daylight.xkqwl"
        }
      },
      "oauth_client": [
        {
          "api_key": [
            {
              "current_key": "AIzaSyCc2XQDt5VykvEmmBcX7AZ7sPhAMt6nWjs"
            }
          ]
        }
      ],
      "services": {
        "appinvite_service": {
          "other_platform_oauth_client": [
            {
              "package_name": "com.aistudio.daylight.xkqwl"
            }
          ]
        }
      }
    }
  ],
  "configuration_version": "1"
}
```

- `app/build.gradle.kts` & `gradle/libs.versions.toml`

```

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.google.devtools.ksp)
    alias(libs.plugins.roborazzi)
    alias(libs.plugins.secrets)
    alias(libs.plugins.google.services)
}

android {
    namespace = "com.example"
    compileSdk { version = release(36) { minorApiLevel = 1 } }

    defaultConfig {
        applicationId = "com.aistudio.daylight.xkqwlp"
        minSdk = 24
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    signingConfigs {
        create("release") {
            val keystorePath = System.getenv("KEYSTORE_PATH") ?: "${rootDir}/my-upload-key.jks"
            storeFile = file(keystorePath)
            storePassword = System.getenv("STORE_PASSWORD")
        }
    }
}

```

- **Configuration:** google-services.json contains the secure client IDs, API keys, and project numbers required to authenticate the network connection with Google's Firebase servers.
- **Dependencies:** The Gradle files pull in the actual network libraries required to transmit data, specifically firebase-firestore and firebase-auth.

4. System Permissions

File: app/src/main/AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity
            android:name="com.example.MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.MyApplication">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

```

- **Authorization:** Contains the explicit declaration `<uses-permission android:name="android.permission.INTERNET" />`. This is required by the Android OS to grant the application permission to open network sockets and stream data.

5. Data Serialization Models

Files: `app/src/main/java/com/example/model/MenuItem.kt`, `User.kt`, `Reservation.kt`

- **Data Mapping:** They represent the exact payload structure transmitted over the network. They use `id: String` to seamlessly map local objects to unique Firestore document IDs during network serialization and deserialization.

Need for networking

The app needs networking to sync menus, reservations, and user profiles in real-time with the Firebase database.

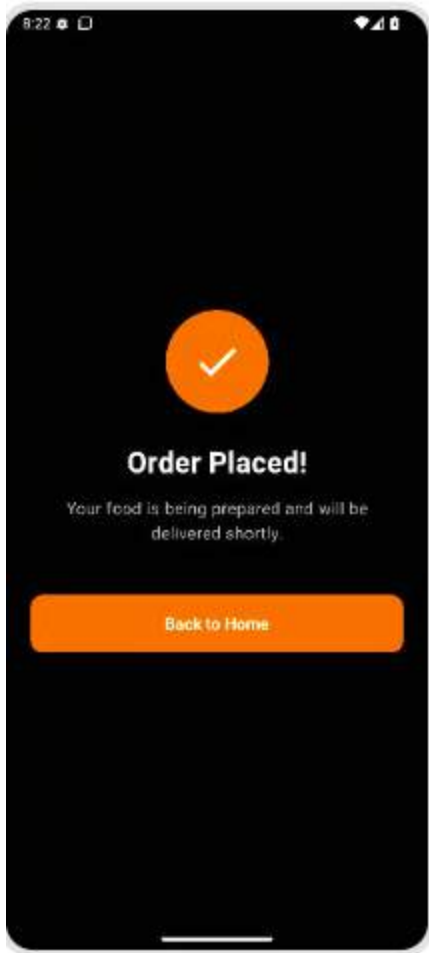
Week 7: Application Architecture

- **UI Layer (View):** Built entirely with **Jetpack Compose**, the UI is declarative and highly modular. Screens (such as `HomeScreen`, `AuthScreens`, and `AdminDashboard`) observe state changes and automatically recompose. Navigation is handled seamlessly using Jetpack Navigation Compose.
- **State Management (ViewModel):** The `MainViewModel` acts as the central hub for presentation logic. It manages the application's lifecycle-aware data using Kotlin `StateFlow`, exposing the current state (like the active user, menu items, or cart contents) to the UI layer while keeping the UI free of business logic.
- **Asynchronous Operations: Kotlin Coroutines** are used throughout the `ViewModel` and `Data` layers to handle background tasks—such as network requests and database queries—without blocking the main user interface thread.
- **Data Layer (Repository):** The `AppRepository` abstracts all data sources. It provides a clean API for the `ViewModel` to request data, ensuring the rest of the application doesn't need to know how or where data is stored.
- **Backend & Persistence:** The app utilizes **Firebase Firestore** as its remote NoSQL database backend. It securely manages user authentication structures and utilizes real-time snapshot listeners to instantly sync live data (like menu items and reservations) across the application.

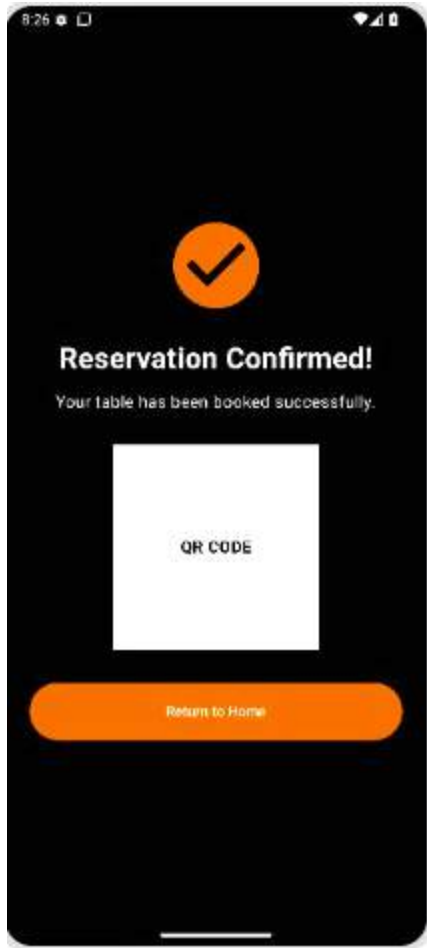
Week 8: Event Handling



On inputting the wrong credentials an error message with the message “invalid credentials” is displayed



Once you place an order a message showing “Order Placed” is displayed



On reserving a table a message with the confirmation of reservation is displayed on your screen



On placing pointer on a button the button is focused as shown in the picture above button focused has a light orange colour

Week 9: Device Features and Sensors

Same as week 1

Week 10: Notifications and Background Services

Same as week 1

Week 11: Mobile Security

Same as week 1

Week 12: Testing and Debugging

Same as week 1

Week 13: Application Deployment

Same as week 1

Week 14: Final Project Presentation

Same as week 1

Semester Project Tracking

Project Title

Project Objectives

Project Requirements

Development Milestones

Testing Results

Lessons Learned

Future Improvements

End of Semester Reflection

Summary of Skills Acquired

Most Interesting Topic

Major Challenges Faced

How Challenges Were Overcome

Professional Growth Achieved

Overall Course Experience

Final Approval

Student Name: _____

Student Signature: _____

Date: _____

Lecturer Name: _____

Lecturer Remarks: _____

Date: _____